TECNICATURA UNIVERSITARIA
EN PROGRAMACIÓN
A DISTANCIA

Trabajo Práctico Semana 1: Gobernanza y Definición de Terminado (DoD)

Objetivo: Establecer los controles de calidad y la estructura de gobernanza del repositorio para garantizar un desarrollo profesional.

Redacción de la DoD: Redacta una "Definition of Done" (Definición de Terminado) formal para tu equipo de trabajo. Debes incluir los 6 criterios obligatorios: integridad de compilación, validación de pruebas automatizadas, umbrales mínimos de cobertura de código, revisión por pares obligatoria, gobernanza de estilo y sustento de la documentación técnica.

La presente Definition of Done es un acuerdo de ingeniería explícito, formal, público y vinculante del equipo de desarrollo. Funciona como una compuerta estricta: ningún incremento de código (feature, corrección o refactorización) puede integrarse a la rama principal (main) si no cumple la totalidad de los criterios que se detallan a continuación. Es global y técnica: se aplica por igual a todas las tareas del sprint, a diferencia de los Criterios de Aceptación, que son específicos y funcionales para cada historia de usuario.
Criterios obligatorios

Criterio 1. Integridad de compilación

- El código nuevo o modificado debe compilar de forma limpia en el entorno local del desarrollador y en el pipeline de integración.
- No se tolera ningún error de compilación ni advertencias (*warnings*) críticas que apunten a futuras fallas de tipado, memoria o uso de APIs obsoletas.
- *Evidencia:* log de compilación sin errores ni warnings críticos adjunto al Pull Request.

Criterio 2. Validación de pruebas automatizadas

- El 100% de la suite de pruebas unitarias y funcionales asociadas al componente debe ejecutarse con resultado exitoso (en verde). Un solo test fallido veta automáticamente la integración.
- Las pruebas siguen el patrón AAA (Arrange, Act, Assert) y contemplan casos de borde (nulos, cadenas vacías, valores fuera de rango) además del camino feliz.
- Las pruebas son independientes, repetibles y se ejecutan de forma rápida (principios F.I.R.S.T.).
- *Evidencia:* reporte de ejecución de pruebas (identificador, fecha, entorno, resultado pass/fail y tiempo en ms) y logs del framework de testing.

Criterio 3. Umbrales mínimos de cobertura de código

- Al menos el 80% de las nuevas líneas de código debe estar cubierto por pruebas automáticas (*statement coverage*).
- Para la lógica con condicionales (if, else, switch, operadores ternarios) se exige además que cada rama tenga al menos un caso que la fuerce a verdadera y otro a falsa (*branch coverage*), porque una cobertura alta de líneas no garantiza la ausencia de bugs ante casos de borde.
- *Evidencia:* informe de cobertura generado por la herramienta del proyecto, en verde.

Criterio 4. Revisión por pares obligatoria (Code Review)

- Otro miembro técnico del equipo (distinto del autor) debe revisar el diff y aprobar el Pull Request.
- El revisor audita que la solución no incorpore malas prácticas, respete los patrones arquitectónicos adoptados y no incremente innecesariamente la complejidad (por ejemplo, complejidad ciclomática elevada o código duplicado).

- Los comentarios, debates de diseño y la aprobación final quedan registrados en el Pull Request como evidencia histórica.
- *Evidencia:* aprobación explícita del revisor en el Pull Request.

Criterio 5. Gobernanza de estilo

- El código debe pasar sin excepciones el *formatter* (por ejemplo Black para Python o Prettier para JavaScript/TypeScript) y el *linter* con las reglas preconfiguradas del proyecto.
- Esto garantiza un estilo uniforme e indistinguible entre desarrolladores y detecta variables o importaciones sin uso, llamadas a funciones inexistentes y patrones inseguros antes de ejecutar el programa.
- La verificación se automatiza mediante un *pre-commit hook* que ejecuta formatter, linter y pruebas unitarias; si cualquiera falla, el commit se aborta.
- *Evidencia:* logs del pre-commit hook con formatter y linter exitosos.

Criterio 6. Sustento de la documentación técnica

- Todo cambio que altere contratos de comunicación debe documentarse de forma obligatoria.
- Se documentan las funciones de alta complejidad, se actualizan los esquemas de base de datos y se regeneran las especificaciones de las APIs (Swagger/OpenAPI).
- Si el cambio no afecta contratos ni lógica compleja, el PR debe declararlo explícitamente ("Sin impacto documental").
- *Evidencia:* archivos de documentación actualizados dentro del mismo Pull Request.

2. **Configuración de Trazabilidad:** Diseña una plantilla de incidencia (Issue Template) profesional en formato Markdown. Asegúrate de que incluya los campos necesarios para eliminar la ambigüedad, como: título, descripción técnica del problema, pasos exactos para reproducir el fallo, comportamiento esperado frente al observado y espacio para evidencias adjuntas.

```
---
name: Reporte de defecto (Bug)
about: Reportar un comportamiento anómalo del sistema
title: "BUG: [breve resumen del fallo]"
labels: bug
assignees: ''
---

## Descripción técnica del problema
<!-- Resumen técnico y detallado del comportamiento anómalo detectado. -->

## Entorno de ejecución
- Sistema operativo:
- Versión del software / commit:
- Rama o fork:
- Versión de lenguaje / dependencias relevantes:

## Pasos exactos para reproducir
<!-- Lista ordenada y numerada de acciones que disparan la falla de forma determinista. -->
1.
2.
3.

## Comportamiento esperado
<!-- Qué debió hacer el sistema según las especificaciones técnicas. -->

## Comportamiento observado
<!-- Qué hizo realmente el sistema: error de consola, congelamiento, desborde de memoria, etc. -->

## Evidencias adjuntas
<!-- Capturas de pantalla, logs recuperados localmente o reportes de consola. -->
- [ ] Captura de pantalla
- [ ] Log / stack trace
```



- [ ] Reporte de pruebas

Severidad

- [ ] Crítica (sistema caído o pérdida de datos)

- [ ] Alta (funcionalidad principal afectada)

- [ ] Media (afecta un flujo secundario)

- [ ] Baja (cosmético o menor)

Checklist previo al envío

- [ ] Verifiqué que el problema no está reportado en otro issue

- [ ] El problema es reproducible siguiendo los pasos indicados

3. **Matriz de Categorización:** Propón un sistema de etiquetas (Labels) para gestionar el repositorio. Debes definir y justificar técnicamente al menos cuatro categorías (como *bug*, *technical-debt*, *documentation* y *feature*) que permitan auditar el estado de cumplimiento de la DoD antes de integrar código.

Label	Uso	Justificación técnica	Relación con la DoD
bug	Exclusivamente fallos lógicos o funcionales confirmados que rompen un comportamiento esperado	Separa los defectos reales de las mejoras; permite medir densidad de defectos y priorizar correcciones	Exige test de regresión en verde (criterio 2) y cobertura del caso corregido (criterio 3)
technical-debt	Refactorizaciones, código de alta complejidad o desvíos de estilo detectados por linters o SonarQube	Hace visible la deuda técnica y evita que se acumule sin gestión; controla complejidad ciclomática y duplicación	Criterios 4 (revisión por pares) y 5 (gobernanza de estilo); las pruebas existentes actúan como red de seguridad
documentation	Actualización de contratos de API, diagramas de arquitectura, esquemas de base de datos o especificaciones Swagger/OpenAPI	Mantiene sincronizados los contratos con el código, lo que evita fallas de integración entre componentes	Criterio 6 (documentación técnica)
feature	Desarrollo de nuevas capacidades lógicas del negocio	Diferencia el valor nuevo del mantenimiento y se vincula con los Criterios de Aceptación de la historia de usuario	Cumplimiento completo de los seis criterios, con cobertura ≥ 80% de líneas nuevas